FACULTEIT INGENIEURSWETENSCHAPPEN
EN ARCHITECTUUR

Profileren van meerdradige applicaties met PICS

Project in het kader van het Vakoverschrijdend Project, Groep BC01

Promotor: Prof. dr. ir. Lieven Eeckhout

Begeleiders: ir. Ge Liu
dr. ir. Jaime Roelandts

Studenten: Tobe De Brabander tobe.debrabander@ugent.be
Jaco Lagrange jaco.lagrange@ugent.be
Louis Verstraeten louis.verstraeten@ugent.be

Academiejaar: 2025–2026

UNIVERSITEIT
GENT

Inhoud

1 Inleiding **3**

2 Context **4**

2.1 Out-of-order execution 4

2.1.1 Parallellisme op instructieniveau 4

2.1.2 De out-of-order pijplijn 5

2.1.3 Onderbrekingen van de ideale instructiestroom 6

2.2 Per-Instruction Cycle Stacks 6

2.2.1 Correcte assignatie 7

2.2.2 Componenten bij dispatch 7

2.2.3 Componenten bij commit 7

2.2.4 Tijdsproportioneel 8

2.3 Sniper simulator 8

3 Doelstellingen **9**

4 Planning **9**

5 Technische bespreking **10**

5.1 Latentie 10

5.2 Implementatie van PICS bij dispatch 10

5.2.1 Base 11

5.2.2 Frontend stall, Backend stall en Misprediction 11

5.3 Implementatie van PICS bij commit 14

5.3.1 Compute 15

5.3.2 Stalled 15

5.3.3 Drained en Flushed 15

6 Invarianten **16**

6.1 Totaal aantal cycli 16

6.2 Base 17
6.3 Frontend stall en Drained 18
6.4 Misprediction 19
6.5 Backend stall en stalled 20

7 Resultaten **21**
7.1 Optimalisatie van PSum 22
7.2 Optimalisatie van AA 24

8 Reflectie **26**

9 Openstaande problemen **26**
9.1 Verloren instructies 27
9.2 TLB misses 27
9.3 DVFS 27

10 Besluit **28**

1 Inleiding

Amdahl's law toont aan dat het relatieve belang van sequentiële code toeneemt naarmate de parallelle code versneld kan worden op gespecialiseerde acceleratoren. Met een toename in de adoptie en mogelijkheden van deze hardware neemt de impact van niet-versnelbare code op de totale prestatie van een applicatie toe. Softwareontwikkelaars zullen in de toekomst dus steeds aandacht moeten besteden aan het optimaliseren van de CPU-prestatie.

Daarvoor is inzicht nodig in waar de processorpijplijn tijd verliest bij de uitvoering van individuele instructies. Dit project focust op het profileren van meerdradige applicaties in de Sniper softwaresimulator, wat als doel heeft softwareontwikkelaars in staat te stellen om de CPU-prestatie van programmacode te visualiseren, om zo gerichte optimalisaties door te kunnen voeren. Het maakt gebruik van een recent voorgestelde profileringsmethode, de *Per-Instruction Cycle Stacks (PICS)*. Om een uitgebreid profiel van de applicatie te verkrijgen, wordt het gedrag per instructie op twee plaatsen in de processorpijplijn onderzocht. Er wordt onderzocht hoe profileringsinformatie verzameld kan worden in de simulator voor applicaties die uitgevoerd worden op zowel enkele als meerdere cores - variërend van één tot honderden - met als doel inzicht te verkrijgen in de invloed van het gedrag van instructies in de processorpijplijn op de uiteindelijke uitvoeringstijd van een applicatie.

Alle code die gebruikt werd voor de resultaten is beschikbaar op GitHub[1]. De broncode van de *Sniper* simulator is inbegrepen als submodule.

[1] `https://github.com/louisverst/VOP-sniper`

2 Context

In deze sectie wordt de breedere context waarin dit project kadert uiteengezet. Eerst wordt een fundamenteel concept uit het ontwerp van moderne, *superscalaire* microprocessors besproken, namelijk *out-of-order (OoO) execution*. Daarna worden ook de *PICS* en de *Sniper* simulator toegelicht.

2.1 Out-of-order execution

Een *superscalaire* microprocessorkern is een kern die meer dan één instructie per klokcyclus kan uitvoeren. Om deze capaciteit in de praktijk te realiseren, wordt in de meeste moderne processorkernen gebruik gemaakt van *OoO execution*. Instructies worden uitgevoerd los van hun volgorde in de programmacode - de *programmavolgorde*. Daarentegen wordt de semantiek van de sequentiële uitvoering van instructies, zoals die in de programmacode staat, wel bewaard voor de bovenliggende software. Dit stelt programmeurs in staat uit te gaan van sequentiële instructies om makkelijker over programmacode te kunnen redeneren, terwijl de processorkern instructies kan uitvoeren in een volgorde die de doorvoer verhoogt. Instructies krijgen in *programmavolgorde* een slot gealloceerd in de *reorder buffer (ROB)*, dit heet *dispatch*. De *ROB* is een hardwarestructuur die de boekhouding van de aanwezige instructies doet in een *First-In, First-Out (FIFO)* wachtrij. Nadat een instructie *dispatcht*, kan deze uitgevoerd worden in een volgorde die de processorpijplijn in staat stelt klokcycli maximaal te benutten. Na uitvoering wordt het effect van de instructies terug in *programmavolgorde* zichtbaar gemaakt, wat *commit* heet. Een kern die gebruikmaakt van *OoO execution* wordt een *OoO* kern genoemd.

2.1.1 Parallellisme op instructieniveau

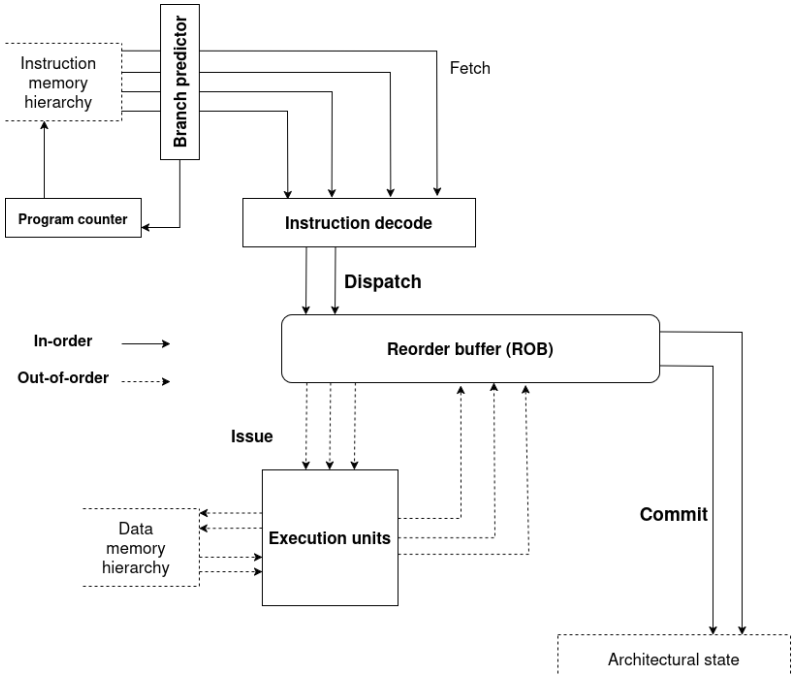
Om meerdere instructies per cyclus uit te kunnen voeren, moet de kern al zeker meerdere instructies per cyclus uit het geheugen halen - d.i. *fetchen* - en moet de hardware nodig om een instructie uit te voeren - zogenaamde *execution units* - meerdere malen aanwezig zijn. Daarnaast moet er genoeg *parallellisme* bestaan tussen de aanwezige instructies. Om efficiënt gebruik te maken van cycli, moeten er steeds instructies aanwezig zijn in de *ROB* die niet afhankelijk zijn van andere, niet reeds uitgevoerde instructies. Een instructie die het resultaat nodig heeft van een eerdere instructie, moet immers wachten tot dat resultaat beschikbaar is, waardoor mogelijks een *execution unit* cycli lang niets doet, wat een opportuniteitskost is.

Om elke cyclus optimaal te benutten moeten er steeds uitvoerbare instructies zijn, wat beperkt wordt indien instructies strikt sequentiëel, in *programmavolgorde* uitgevoerd worden. Het is deze beperking die *OoO execution* probeert te omzeilen. Door toe te laten dat instructies uitgevoerd worden in een andere volgorde dan de *programmavolgorde*, en dus

meer instructies te beschouwen voor uitvoering, wordt het waarschijnlijker dat er steeds een uitvoerbare instructie beschikbaar is.

2.1.2 De out-of-order pijplijn

In deze sectie wordt in meer detail besproken hoe een instructie, van *fetch* tot aan *commit*, verwerkt wordt. Figuur 1 toont een geabstraheerde, schematische voorstelling van een moderne *OoO* pijplijn. De pijlen in stippellijn duiden op operaties die *out-of-order* kunnen, zoals het naar de *execution units* brengen - wat *issue* genoemd wordt. De pijlen met een volle lijn zijn operaties die *in-order* moeten gebeueren. De *dispatch* en de *commit* van instructies gebeurt dus altijd *in-order*.



Figuur 1: Schematische weergave van een out-of-order pijplijn. (Gebaseerd op [1])

Instructies worden eerst uit het geheugen opgehaald - *fetch*, waarna ze gedecodeerd worden. Bij *complex instruction set computing (CISC)* architecturen (zoals *x86*) kunnen instructies meer dan één operatie uitvoeren. Een instructie kan bijvoorbeeld zowel een waarde uit het geheugen halen en ook met die waarde een berekening uitvoeren. Na het *decoderen* van de instructie onstaan twee *micro-operaties (µops)*. Eén om naar het geheugen te gaan, en één om de berekening uit te voeren. Deze twee *µops* worden afzonderlijk in de *ROB* gebracht. Een karakteriserende eigenschap van *reduced instruction set computing (RISC)* architecturen (zoals ARM architecturen) is dat elke instructie slechts één operatie in de pijplijn uitvoert. Bijvoorbeeld enkel *load*-instructies gaan naar het geheugen, terwijl bij *CISC* ook een arithmetische instructie naar het geheugen kan gaan.

Het ophalen van instructies gebeurt dus in *programmavolgorde*. Op de figuur staan vier pijlen afgebeeld, wat zou impliceren dat er elke klokcyclus vier instructies uit het geheugen

gehaald worden, het maximaal aantal instructies dat uit het geheugen kan gehaald worden verschilt echter per *microarchitectuur*.

Na *dispatch* staan de slots in de *ROB* corresponderend met de *gedispatchte µops* in *programmavolgorde*. De *µop* zal uitgevoerd worden - *issue* - vanaf alle afhankelijke *µops* klaar zijn en er een *exeuction unit* beschikbaar is. Net als dat er bij *fetch* elke cyclus meerdere instructies opgehaald kunnen worden, kunnen er ook elke cyclus meerdere *µops dispatchen*. Hoeveel er maximaal *dispatchen*, is *microarchitectuur*-afhankelijk, en niet noodzakelijk gelijk aan het aantal instructies dat *gefetcht* worden.

Elke cyclus probeert de hardware de oudste *µop* te *committen*. Deze staan in *programmavolgorde* in de *ROB* en *committen* bijgevolg dus ook in *programmavolgorde*. Na *commit* is het effect van de instructie zichtbaar wordt in de *architecturale staat*. Dit wil zeggen dat het resultaat bijvoorbeeld zichtbaar is in een register, of in het geheugen. Alles wat voor *commit* gebeurt is speculatief, bij een foutieve *sprongvoorspelling* worden er mogelijks *µops* uitgevoerd waarvan het resultaat later wordt tenietgedaan. Wanneer een *µop commit*, komt het bijhorende slot in de *ROB* vrij.

2.1.3 Onderbrekingen van de ideale instructiestroom

In het ideale, hypothetische geval worden elke cyclus het maximaal aantal instructies opgehaald uit het geheugen, *dispatchen* er ook het maximaal aantal *µops*, dit heet de *dispatchbreedte* of het aantal *dispatchslots*. Analoog *committen* er in het ideale geval ook het maximaal aantal *µops* per cyclus. Men spreekt van de *commitbreedte* en het aantal *commitslots*. Dit ideale scenario gebruikt de onrealistische veronderstelling dat elke instructie na één cyclus beschikbaar is, dat elke instructie in één cyclus kan uitgevoerd worden en dat er geen afhankelijkheden tussen instructies zijn. In realiteit kan men deze aannames niet maken en zijn er talloze redenen waarom de pijplijn kan *stallen*, wat de stroom van instructies onderbreekt. De *PICS*, die hieronder worden toegelicht, zullen deze opdelen in vier componenten.

2.2 Per-Instruction Cycle Stacks

De recent gepubliceerde papers *TEA: Time-Proportional Event Analysis* [2] en *Chips Need DIP: Time-Proportional Per-Instruction Cycle Stacks at Dispatch* [3] introduceren de *PICS*. *PICS* proberen de uitvoering van computerprogramma's te visualiseren, zodat kan worden geïdentificeerd welke instructies het meeste tijd kosten en waaraan die tijd precies wordt besteed. *PICS* kunnen opgesteld worden zowel aan de *dispatch*- als aan de *commit*-fase. Echter zal blijken dat ze vaak een complementair beeld geven. Hierdoor is het belangrijk om beide te implementeren. Dit zal geïllustreerd worden aan de hand van een aantal voorbeelden in sectie 7.

2.2.1 Correcte assignatie

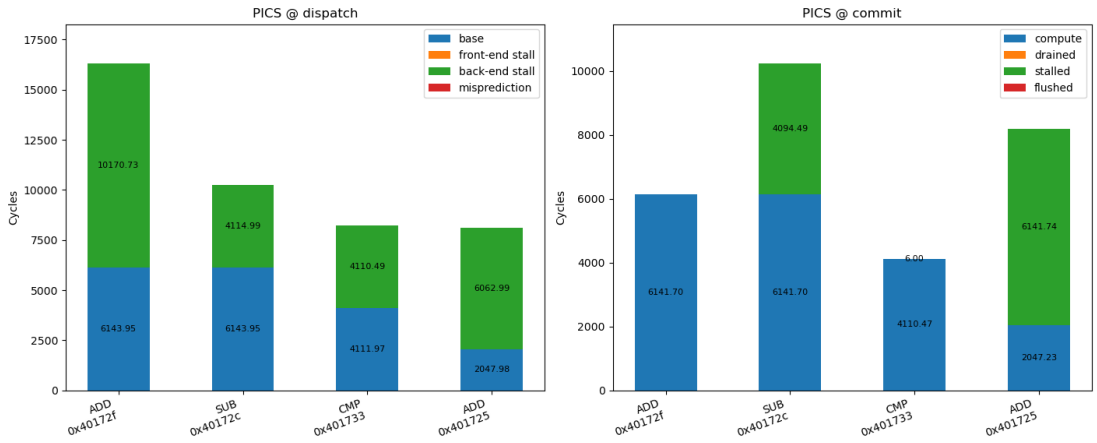
Elke cyclus die een kern spendeert aan het uitvoeren van een programma moet niet alleen geattribueerd worden aan de statische instructie uit de programmacode die op dat moment de kern bezet, maar ook aan de juiste component van de bijhorende *PICS*. Een component correspondeert met een onderliggende oorzaak van prestatieproblemen. Dit stelt ontwikkelaars in staat gerichte optimalisatiemogelijkheden te identificeren. Figuur 2 toont een voorbeeld van *PICS* voor vier instructies, zowel bij *dispatch* als bij *commit*, waar twee componenten duidelijk domineren. Welke componenten er beschouwd worden, wordt nu toegelicht.

2.2.2 Componenten bij dispatch

- **Base**: Er treden geen problemen op, alle *dispatchslots* worden ingenomen.
- **Frontend stall**: Niet alle *dispatchslots* kunnen ingenomen worden doordat het ophalen van een instructie langer duurt door bijvoorbeeld een *I-cache miss* (*Instruction-cache miss*) of *TLB-miss*.
- **Backend stall**: Niet het maximaal aantal *µops* kunnen *dispatchen* doordat de *ROB* volzet is. Dit is het gevolg van te veel *µops* die nog bezig zijn met uitvoeren, of wachten op afhankelijke instructies.
- **Misprediction**: Er werd een sprong verkeerd voorspeld. Daardoor komen er verkeerde instructies in de pijplijn en gaat er tijd verloren.

2.2.3 Componenten bij commit

- **Compute**: De oudste *µop* in de *ROB* is klaar om te *committen*, en de *commitfase* kan zonder beperkingen doorgaan.
- **Drained**: De *ROB* is leeg, dus er is geen oudste *µop* om te *committen*.
- **Stalled**: De oudste *µop* in de *ROB* is nog niet uitgevoerd. Daardoor kan deze niet *committen*.
- **Misprediction**: De *ROB* is opnieuw leeg, maar dit keer omdat deze leeggemaakt is doordat deze gevuld geraakt is met verkeerde *µops* ten gevolge van een foute sprongvoorspelling.



Figuur 2: Voorbeeld van PICS bij dispatch en bij commit voor vier instructies.

2.2.4 Tijdsproportioneel

In de uiteindelijke profileringsinformatie is een correcte toewijzing van cycli aan statische instructies van groot belang (hierna kortweg instructies genoemd). Het is cruciaal dat deze profileringsinformatie tijdsproportioneel is: het belang van een instructie in het uiteindelijke profiel moet evenredig zijn met de relatieve bijdrage van die instructie aan de totale uitvoeringstijd. Dit vereist een nauwkeurige toewijzing - zonder cycli toe te kennen aan instructies die niet hebben bijgedragen aan de uitvoeringstijd, en zonder cycli te onthouden aan instructies die dat wel hebben gedaan.

2.3 Sniper simulator

De *Sniper* simulator [4] is een softwaregebaseerde simulator voor de *x86 architectuur* die parallelle programma's kan simuleren. De simulator probeert de afweging te maken tussen simulatiesnelheid en nauwkeurigheid.

Om maximale nauwkeurigheid te bereiken, zou de simulatie cyclus per cyclus moeten gebeuren. Dit zou echter ten koste gaan van de simulatiesnelheid, wat het praktisch gebruik niet werkbaar zou maken. Eén van de technieken om beiden te combineren is *intervalsimulatie* [5]. De uitvoering van een programma door een processorkern kan gezien worden als een stroom van instructies die onderbroken wordt door *miss events* zoals *cache misses*, *sprongvoorspellingen*, etc. De simulator kan dus werken op een hoger abstractieniveau door tussen deze *miss events* te springen en ertussen een constante stroom van instructies aan te nemen. Alhoewel deze techniek de eerder vermelde voordelen heeft, bemoeilijkt het de implementatie van *PICS*'s.

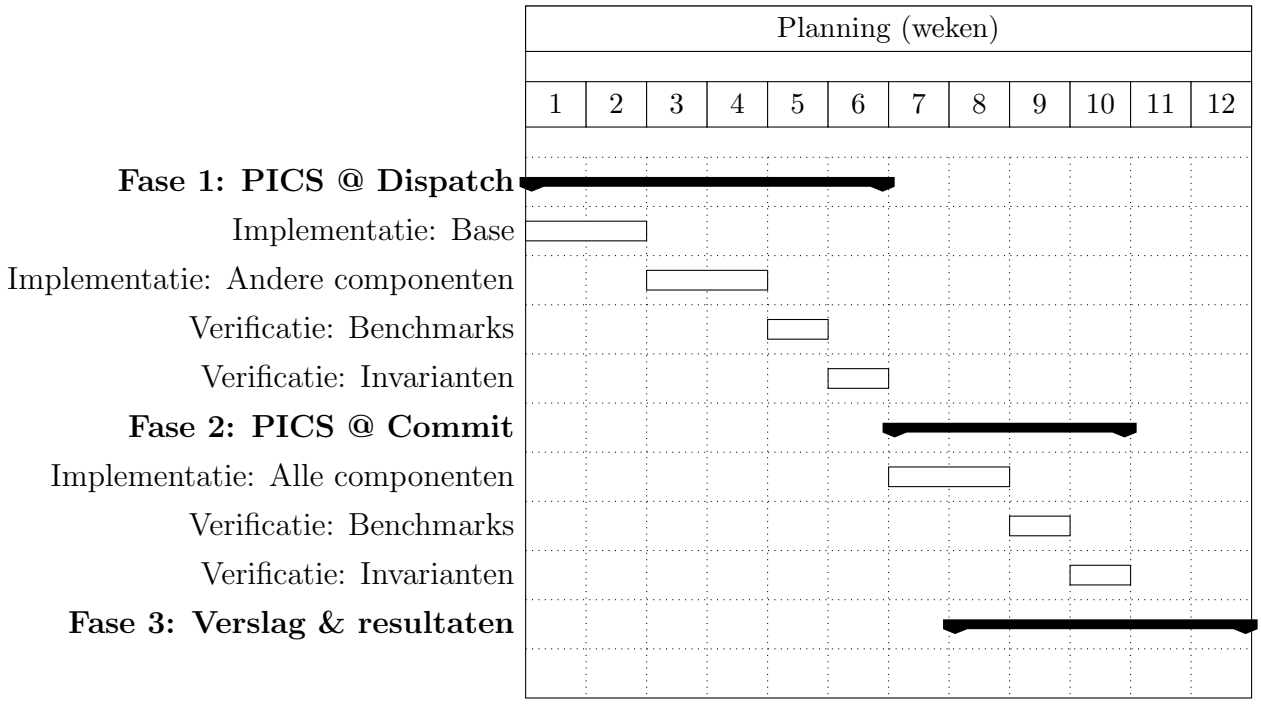
3 Doelstellingen

Er bestaan reeds verschillende profileringsmethoden ([6], [7], [8], en nog meer) die informatie kunnen genereren over de prestatie van applicaties, maar deze voldoen niet altijd tegelijkertijd aan beide vereisten voor een goede profileringsmethode (tijdsproportioneel en correcte assignatie). Daarnaast is het duidelijk dat profileren zowel bij *dispatch* als bij *commit* belangrijk is, aangezien dit vaak een complementair beeld oplevert.

Het doel van dit project is dan ook om *PICS* te implementeren op beide plaatsen in de pijplijn binnen de *Sniper* simulator, en dit te valideren aan de hand van gerichte *microbenchmarktests* (zie sectie 6, de definitie van een *microbenchmark*). De geïmplementeerde methoden zullen vervolgens worden geëvalueerd en bijgestuurd op basis van de verkregen resultaten.

4 Planning

De planning is weergegeven in Figuur 3. Initieel was voor Fase 2 meer tijd voorzien, maar dankzij de kennis over de simulator en de aanpak om *PICS* te implementeren, opgedaan tijdens Fase 1, konden bij *PICS @ Commit* alle componenten tegelijk aangepakt worden, al is de inhoudelijke aanpak niet dezelfde. Hierdoor kon er vroeger worden begonnen aan het verslag en aan optimalisaties van enkele *microbenchmarks*, waarover meer in de resultaten.



Figuur 3: Projectplanning

5 Technische bespreking

Deze sectie zet de belangrijkste ideeën van de implementatie binnen de software uiteen. Er wordt een hoog niveau beeld gegeven van hoe de simulatie te werk gaat en hoe de *PICS* geïmplementeerd worden.

5.1 Latentie

Aangezien de *Sniper* simulator gebruik maakt van *intervalsimulatie*, wordt er continu een latentie berekend tot de volgende gebeurtenis. Dit zogenaamd *next event* bepaalt in de hoofdlus van de simulator het volgende relevante tijdstip, waarna de simulatie in één stap naar dat punt vooruit springt. Op dat moment wordt voor elke pijplijnfase opnieuw geëvalueerd of verdere voortgang mogelijk is. De pijplijn zelf wordt gemodelleerd aan de hand van drie hoofdfases - *dispatch*, *issue* en *commit* - waarbij elke fase met een eigen set slots werkt en, op basis van de huidige toestand, bepaalt wanneer een volgende gebeurtenis kan plaatsvinden. Zolang er dus in minstens één van deze fases een slot is dat klaar is om te verwerken, zal de simulator blijven doorgaan. De latentie is dan gelijk aan één, want er wordt gesprongen naar de volgende cyclus.

Binnen dit model is het belangrijk om latenties correct toe te wijzen aan de juiste instructies en om onderscheid te maken tussen de verschillende oorzaken van vertragingen. Zo zal bijvoorbeeld bij een *cache-miss* een penalty worden toegevoegd aan de latentie, waardoor de volgende gebeurtenis later plaatsvindt. Deze latenties omvatten steeds alle cycli, zelfs wanneer het volgende event zich al in de eerstvolgende cyclus voordoet. Daarom is het idee om op het moment dat een vertraging start te markeren welke instructie hiervoor verantwoordelijk is, en vervolgens alle latenties tot het einde van die vertraging consequent aan deze instructie toe te wijzen.

Belangrijk om op te merken is dat als de latentie gelijk is aan één we in een speciaal geval terecht komen waar ofwel in de huidige cyclus alles correct verloopt ofwel er een vertraging gestart is. Hier zal extra aandacht aan moeten worden besteed en er zal in dit specifieke geval geen attributie van cycli zijn a.d.h.v. het latentie mechanisme.

In wat volgt zal er verder worden ingegaan op de implementatie van *PICS* bij *dispatch* en *commit*.

5.2 Implementatie van PICS bij dispatch

Het doel van de implementatie bij *dispatch* is om de latentie van elk *dispatchslot* (1/*dispatchbreedte*) toe te wijzen aan de instructie die overeenkomt met de μop verantwoordelijk voor de toestand van dat slot (zie sectie 2.2.2). Op deze manier wordt elke *dispatchcyclus* volledig geattribueerd: ofwel aan één instructie, ofwel opgesplitst over meerdere instructies.

In *Sniper* wordt het mechanisme van *dispatchslots* uitgebreid. Voor elke *µop*, toegewezen aan een *slot* wordt gecontroleerd of de backend voldoende hardware vrij heeft om de *µop* te ontvangen. Daarnaast worden *I-cache misses* en spronginstructies ook expliciet behandeld in de *dispatchfase*. Opdat een *µop gedispatcht* kan worden in *Sniper*, wordt er gecontroleerd in welke van de vier toestanden zijn toegewezen *dispatchslot* zich bevindt. O.b.v. die toestand wordt er bepaald wat er met de *µop* en de *dispatchflow* gebeurt. Er zijn twee mogelijkheden:

1. De *µop* wordt *gedispatcht* en *dispatchen* blijft doorgaan. Het slot bevindt zich in de *base* toestand.
2. De *µop* wordt al dan niet *gedispatcht*, maar *dispatchen* wordt onderbroken. Het slot bevindt zich in één van de andere drie toestanden.

Deze twee gevallen worden hieronder verder toegelicht.

5.2.1 Base

Dit beschrijft de normale werking van de *dispatchfase*. In dit geval ondervindt de *µop* geen *I-cache miss*, is er voldoende ruimte beschikbaar in de backend om nieuwe *µops* te ontvangen, en betreft het geen foutief voorspelde spronginstructie. De *µop* kan bijgevolg worden *gedispatcht* en de *dispatchfase* blijft actief.

Bij *dispatch* wordt een fractie van de cyclus, namelijk $1/\mathit{dispatchbreedte}$, toegewezen aan de *base* component van de instructie die overeenkomt met deze *µop*. Elke *gedispatchte µop* draagt dus precies $1/\mathit{dispatchbreedte}$ bij aan deze *base* component. Aangezien een statische instructie meerdere keren kan worden uitgevoerd en kan bestaan uit meerdere *µops*, geldt uiteindelijk:

$$\mathrm{PICS_b} = (N_\mathrm{e} \cdot N_\mu) \cdot \frac{1}{\mathit{dispatchbreedte}} \tag{1}$$

Hierbij stellen N_e en N_μ respectievelijk het aantal uitvoeringen van de statische instructie en het aantal *µops* waaruit deze instructie bestaat voor. $\mathrm{PICS_b}$ staat voor de *base* component van de statische instructie.

Na het *dispatchen* van de *µop* en het toewijzen van de fractie van de *dispatchcyclus* aan de *base* component, selecteert de *dispatchfase* de volgende *µop* en wordt het proces herhaald.

5.2.2 Frontend stall, Backend stall en Misprediction

In dit geval ondervindt de *µop* één van de drie mogelijke problemen die de *dispatchfase* zal onderbreken: een *I-cache miss* (*frontend stall*), een hardware conflict in de backend (*backend stall*), of een foutief voorspelde spronginstructie (misprediction). Voor elke oorzaak

wordt hetzelfde basismechanisme gehanteerd. Wanneer een *µop* een *stall* veroorzaakt, wordt deze samen met de *stall* oorzaak opgeslagen in een datastructuur. Vanaf dat moment wordt de *dispatchfase* volledig geblokkeerd en kunnen er geen nieuwe *µops* meer *gedispatcht* worden. Alle *dispatchslots* blijven gedurende deze periode ongebruikt en hun latentie moet bijgevolg geattribueerd worden.

Hoewel *dispatch* stilvalt, blijft de hoofdlus van de simulator verder lopen, aangezien andere fasen eventueel nog nuttig werk kunnen verrichten. In elke iteratie bepalen de verschillende fasen hun volgende event, wat aangeeft tot welk tijdstip de simulator vooruit kan springen. Elke cyclus die tijdens deze periode wordt overgeslagen, zal worden toegewezen aan de *PICS* component die overeenkomt met de *stall* oorzaak van de instructie waartoe de *µop* in de datastructuur behoort. Op deze manier worden alle cycli waarin de *dispatchfase* onderbroken is volledig geattribueerd aan één statische instructie. Zodra de *stall* is afgelopen en *dispatch* opnieuw kan verdergaan, wordt de *µop* uit de datastructuur verwijderd zodat enkel en alleen de *stall* latentie geattribueerd wordt.

Daarnaast is het belangrijk om op te merken dat ook binnen de cyclus waarin de *stall* optreedt, niet alle *dispatchslots* noodzakelijk benut worden. Hun latentie moet eveneens op correcte manier toegewezen worden. Stel dat *dispatchbreedte* gelijk is aan vier. Indien in een bepaalde cyclus het eerste *dispatchslot* succesvol een *µop dispatcht*, maar het tweede slot een *µop* bevat die een *I-cache miss* veroorzaakte, wordt de *dispatchfase* vanaf dat moment onderbroken. De resterende drie *dispatchslots* in die cyclus blijven ongebruikt. De fractie van de *dispatchcyclus* die overeenkomt met deze ongebruikte *slots* (in dit geval 3/4 van de cyclus) wordt eveneens toegewezen aan de *PICS* component die overeenkomt met de *stall* oorzaak. Samen met de reeds geattribueerde *base* component van het eerste *dispatchslot* wordt zo de volledige *dispatchcyclus* geattribueerd.

Dit mechanisme garandeert dat elke cyclus volledig geattribueerd wordt, wat later geverifieerd zal worden (zie sectie 6). De drie oorzaken voor een *stall* verschillen voornamelijk in hoe en wanneer hun latentie bepaald wordt. In wat volgt zullen de drie gevallen verder genuanceerd worden.

1. **Frontend stall.** Een *frontend stall* wordt veroorzaakt door een *I-cache miss*. In een echte CPU zal een *I-cache miss* in de *fetchfase* ervoor zorgen dat de toevoer van *µops* naar de *dispatchfase* tijdelijk onderbroken wordt. In *Sniper* blijft deze toevoer echter constant. Een *I-cache miss* wordt daarom niet gemodelleerd door het ontbreken van *µops*, maar door het artificieel blokkeren van de *dispatchfase*. Hoewel er dus nog *µops* beschikbaar zijn, wordt *dispatch* expliciet onderbroken om de vertraging ten gevolge van de *I-cache miss* te simuleren. Meer concreet wordt tijdens het *dispatchen* van elke *µop* gecontroleerd of deze onderhevig was aan een *I-cache miss*. Indien dit het geval is, wordt de *dispatchfase* onmiddellijk onderbroken, ongeacht het aantal resterende *dispatchslots* in dezelfde cyclus. De oorzaak hiervan is dat de instructie waartoe de *µop* behoort nog niet beschikbaar is vanuit het instructiegeheugen. In een echte CPU zou het overeenkomstige *dispatchslot* dus leeg zijn, maar door de

constante toevoer in *Sniper* is de *µop* reeds aanwezig. Hoewel de *miss* eerder in de pijplijn - tijdens *decode* - werd gedetecteerd en de bijhorende latentie daar werd bepaald, wordt deze pas zichtbaar in de *dispatchfase*. Vanaf het moment dat de *stall* optreedt, wordt het basismechanisme toegepast: de *µop* en de *stall* oorzaak worden in de datastructuur geplaatst, en de *dispatchfase* blijft geblokkeerd tot het tijdstip waarop de instructie zou terugkeren uit het instructiegeheugen. Dit tijdstip wordt als *next event* teruggegeven aan de hoofdlus. Alle cycli tijdens deze periode worden toegewezen aan de frontend component van de corresponderende statische instructie. Dit omvat zowel volledige cycli zonder *dispatch* als het resterende deel van de cyclus waarin de *stall* initieel optrad. Op deze manier wordt de volledige *miss* latentie correct en volledig geattribueerd.

2. **Backend stall.** Een *backend stall* treedt op wanneer er onvoldoende hardware beschikbaar is in de backend om een *µop* te ontvangen. In *Sniper* kan dit twee oorzaken hebben: de *ROB* is vol, of de *reservation stations* hebben geen vrije plaatsen meer. Tijdens de *dispatchfase* wordt voor elke *µop* gecontroleerd of beide hardwarecomponeenten voldoende capaciteit hebben. Indien één van deze niet vrij is, kan de *µop* niet *gedispatcht* worden. De *µop* die niet *gedispatcht* kan worden, wordt samen met de *stall* oorzaak in de datastructuur geplaatst, waarna de *dispatchfase* wordt onderbroken. In tegenstelling tot een *frontend stall* is het eindtijdstip van de *stall* hier niet op voorhand gekend. Het vrijmaken van hardware in de backend hangt af van de voortgang in de *issue*- en *commitfase*, die *entries* uit de *reservation stations* en de *ROB* verwijderen. Daarom wordt als *next event* een oneindig tijdstip teruggegeven. De voortgang van de simulatie wordt hierdoor volledig bepaald door events in de *issue*- en *commitfase*. De hoofd lus springt telkens naar het eerstvolgende event, waarna opnieuw gecontroleerd wordt of *dispatch* hervat kan worden. Zolang de hardware in de backend niet beschikbaar is, blijft de *µop* in de datastructuur aanwezig en worden alle cycli toegewezen aan de *backend stall* component van de corresponderende statische instructie volgens het basismechanisme. Zodra een event in de backend voldoende hardware vrijmaakt, kan de *µop* alsnog *gedispatcht* worden. Op dat moment wordt deze uit de datastructuur verwijderd en wordt de *dispatchfase* hervat. Op deze manier wordt de volledige *stall* latentie correct en volledig geattribueerd.

3. **Misprediction.** Een *stall* ten gevolge van een foute voorspelling wordt veroorzaakt door een foutief voorspelde spronginstructie. In een echte CPU wordt een sprong *µop* behandeld zoals een gewone *µop*: deze wordt *gedispatcht* samen met *µops* op het voorspelde pad. Indien tijdens de *executefase* blijkt dat de voorspelling fout was, worden alle jongere *µops* *geflusht* en wordt het *fetchen* hervat vanaf het correcte pad. In *Sniper* wordt dit gedrag anders gemodelleerd. Aangezien de simulator kennis heeft van het correcte pad, worden foutief gespeculeerde *µops* niet *gedispatcht*. In plaats daarvan wordt de kost van een foute voorspelling gesimuleerd door de *dispatchfase* tijdelijk te blokkeren. Tijdens *dispatch* wordt gecontroleerd of een *µop*

een foutief voorspelde sprong is. Indien dit het geval is, wordt de *µop* nog *gedispatcht*. Vervolgens wordt deze samen met de *stall* oorzaak in de datastructuur geplaatst en wordt de *dispatchfase* geblokkeerd. De totale *stall* latentie bestaat uit twee componenten. Enerzijds is er de *resolution time*: de tijd tot de sprong effectief uitgevoerd wordt in de *issuefase*. Tijdens deze periode is het eindtijdstip van de *stall* nog niet gekend. Vanaf het moment dat de sprong uitgevoerd wordt, wordt het eindtijdstip wel bepaald. Dit komt overeen met een vaste *flush penalty*, die de tijd modelleert nodig om de pijplijn te herstellen en het correcte pad opnieuw op te starten. Gedurende de volledige *stall* periode worden alle cycli toegewezen aan de *misprediction* component van de corresponderende statische instructie. Dit omvat ook de resterende *dispatchslots* in de cyclus waarin de *stall* initieel optrad. Zodra de *flush penalty* verstreken is, wordt de *µop* uit de datastructuur verwijderd en wordt de *dispatchfase* hervat. Op deze manier wordt de volledige kost van een foutieve sprongvoorspelling correct en volledig geattribueerd.

5.3 Implementatie van PICS bij commit

Het doel van de implementatie bij *commit* is analoog aan dat bij *dispatch* (zie sectie 5.2): de latentie van elk *commitslot* (1/*commitbreedte*) wordt toegewezen aan de instructie die overeenkomt met de *µop* verantwoordelijk voor de toestand van dat *slot* (zie sectie 2.2.3). Op deze manier wordt elke *commitcyclus* volledig geattribueerd: ofwel integraal aan één instructie, ofwel opgesplitst over meerdere instructies.

In tegenstelling tot *dispatch* bevat elk *commitslot* geen *µop*, maar een *entry* in de *ROB*. Een dergelijke *entry* kan een *µop* bevatten, maar kan ook leeg zijn. Om te bepalen of een *entry* vooraan[2] in de *ROB* kan worden verwijderd (d.i. de *µop* kan *committen*), wordt nagegaan in welke van de vier toestanden het bijhorende *commitslot* zich bevindt.

Afhankelijk van deze toestand wordt bepaald wat er met de *entry* en de *commitflow* gebeurt. Er zijn drie mogelijke gevallen:

1. De *entry* bevat een *µop* die kan *committen*. Het *slot* bevindt zich in de *compute* toestand en *committen* gaat verder.

2. De *entry* bevat een *µop*, maar deze kan niet *committen*. Het *slot* bevindt zich in de *stalled* toestand en *committen* wordt onderbroken.

3. De *entry* bevat geen *µop*. Het *slot* bevindt zich in de *drained* of *flushed* toestand en *committen* wordt onderbroken.

Deze drie gevallen worden hieronder verder toegelicht.

[2] d.i. de positie van de oudste *µop* in de *ROB*

5.3.1 Compute

Dit beschrijft de normale werking van de *commitfase*. De *entry* bevat een *µop* die succesvol kan *committen*. Bijgevolg wordt een fractie van de cyclus - namelijk 1/*commitbreedte* - toegewezen aan de *compute* component van de instructie waartoe deze *µop* behoort.

Analoog aan formule (1) geldt uiteindelijk:

$$\mathrm{PICS_c} = (N_\mathrm{e} \cdot N_\mu) \cdot \frac{1}{commitbreedte} \tag{2}$$

Hierbij stellen N_e en N_μ respectievelijk het aantal uitvoeringen van de statische instructie en het aantal *µops* waaruit deze instructie bestaat voor. $\mathrm{PICS_c}$ stelt de *compute* component van de statische instructie voor.

Na het *committen* van de *µop* en het toewijzen van de fractie van de cyclus, wordt de volgende *entry* geselecteerd en wordt het proces herhaald.

5.3.2 Stalled

In dit geval bevat de *entry* een *µop* die nog niet kan *committen* omdat deze nog niet volledig uitgevoerd is. Vanaf het moment dat dit gedetecteerd wordt, wordt de *commitfase* onderbroken en kunnen er geen verdere *µops committen*. Alle *commitslots* blijven gedurende deze periode ongebruikt en hun latentie moet bijgevolg geattribueerd worden. Hiervoor wordt hetzelfde basismechanisme gebruikt als bij *dispatch* (zie sectie 5.2.2). De *µop* wordt samen met de *stall* oorzaak (*stalled*) opgeslagen in een datastructuur. Vanaf dat moment wordt de *commitfase* volledig geblokkeerd. Tijdens elke iteratie van de hoofdlus worden de overgeslagen cycli toegewezen aan de *stalled* component van de corresponderende statische instructie. Net zoals bij *dispatch* wordt ook rekening gehouden met het resterende deel van de cyclus waarin de *stall* initieel optrad. De latentie van alle daaropvolgende *commitslots* in dezelfde cyclus wordt eveneens toegewezen aan de *stalled* component. Zodra de *µop* wel kan *committen*, wordt deze uit de datastructuur verwijderd zodat enkel de *stall* latentie geattribueerd wordt, en wordt de *commitfase* hervat.

5.3.3 Drained en Flushed

In dit geval bevat de *entry* geen *µop*, wat betekent dat de *ROB* leeg is. Als gevolg hiervan wordt de *commitfase* onderbroken totdat er opnieuw *µops* beschikbaar zijn in de *ROB*. Alle *commitslots* blijven gedurende deze periode ongebruikt en hun latentie moet geattribueerd worden. Opnieuw wordt hiervoor hetzelfde basismechanisme gebruikt als bij *dispatch* (zie sectie 5.2.2). Een *µop* wordt geïdentificeerd als verantwoordelijk voor het leeg zijn van de *ROB* en wordt samen met de oorzaak (*drained* of *flushed*) in de datastructuur geplaatst. Alle cycli tijdens deze periode worden toegewezen aan de overeenkomstige component van de statische instructie waartoe deze *µop* behoort. Zodra er opnieuw *µops*

in de *ROB* aanwezig zijn, wordt deze *µop* uit de datastructuur verwijderd en wordt de *commitfase* hervat. Net zoals bij de andere gevallen wordt ook hier rekening gehouden met de resterende *commitslots* in de cyclus waarin deze toestand optreedt.

Het identificeren van de verantwoordelijke *µop* gebeurt als volgt:

1. **Flushed.** Indien de laatst *gecommitte µop* behoort tot een fout voorspelde spronginstructie, wordt deze beschouwd als verantwoordelijk voor het leeg zijn van de *ROB*. Zoals besproken in sectie 3, leidt een foute voorspelling ertoe dat de *dispatchfase* tijdelijk onderbroken wordt. Indien op het moment dat deze *µop commit* de *dispatchfase* nog steeds geblokkeerd is, zijn er geen nieuwe *µops* aangevoerd en is de *ROB* leeg ten gevolge van deze foute voorspelling. In dat geval wordt de latentie toegewezen aan de *flushed* component van de corresponderende spronginstructie.

2. **Drained.** Indien de laatst *gecommitte µop* niet behoort tot een fout voorspelde spronginstructie, wordt gekeken naar de volgende *µop* die *gedispatcht* zal worden. Indien deze *µop* een *I-cache miss* ondervindt, wordt de *dispatchfase* tijdelijk onderbroken, waardoor de toevoer van *µops* naar de *ROB* stilvalt. De *ROB* loopt hierdoor leeg, en deze *µop* wordt beschouwd als verantwoordelijk voor deze toestand. De latentie wordt in dat geval toegewezen aan de *drained* component van de corresponderende instructie.

6 Invarianten

Deze sectie onderbouwt de implementatie van *PICS* aan de hand van invarianten: eigenschappen die steeds gelden voor *PICS*. De invarianten zullen de data gegenereerd door de implementatie van *PICS* vergelijken met de bestaande output van de *Sniper* simulator, en daarmee verzekeren dat alle gegenereerde data consistent is. Deze invarianten zullen toegepast worden aan de hand van *microbenchmarks*. Een *microbenchmark* is een klein programma dat één component van de *OoO* pijplijn belast, zodat het effect ervan geïsoleerd gemeten kan worden. De invarianten vormen geen sluitend bewijs van correctheid, maar een schending ervan zou onmiskenbaar wijzen op een fout in de implementatie.

6.1 Totaal aantal cycli

Per cyclus kent *PICS* de cyclus (of een fractie ervan, zoals in sectie 5.2) toe aan de instructies die in die cyclus *dispatchen*. Elke cyclus wordt volledig verdeeld - er gaat niets verloren en er wordt niets bijgeteld. De totale som van alle toegewezen cycli over alle *PICS* bij *dispatch* moet daarom per definitie gelijk zijn aan de totale gesimuleerde duur van het programma (in cycli). Eenzelfde redenering kan gemaakt worden voor de *PICS* bij *commit*, die onafhankelijk tot dezelfde som moeten leiden.

Tabel 1 toont een selectie van *microbenchmarks*. De kolom ROB-cycli bevat het totaal aantal simulatiecycli zoals gerapporteerd door de *Sniper* simulator. Er wordt echter een correctie uitgevoerd op dit aantal, wat onderbouwd wordt in sectie 9. De kolommen $PICS_d$ en $PICS_c$ bevatten de som van alle *PICS*-waarden bij respectievelijk *dispatch* en *commit*. De bijhorende Δ-kolommen geven het verschil in absolute cycli met de referentie. De Δ ligt bij *commit* hoger doordat de gedetaileerde simulatie afsluit wanneer sommige *µops* nog niet gesimuleerd zijn, zoals verder uitgelegd in sectie 9.

Benchmark	ROB-cycli	$PICS_d$	Δ_d	$PICS_c$	Δ_c
ED1	118.438,73	118.438,70	$-0,03$	118.435,71	$-3,03$
STL2b	295.280,33	295.280,26	$-0,07$	295.277,22	$-3,11$
MCS	245.360,45	245.360,31	$-0,14$	245.357,44	$-3,00$
MI	48.538,89	48.538,75	$-0,14$	48.535,50	$-3,39$
DPcvt	21.845,95	21.845,75	$-0,20$	21.842,72	$-3,23$
EI	7.197,98	7.197,75	$-0,23$	7.194,70	$-3,28$
CCl	5.874,99	5.874,74	$-0,25$	5.871,72	$-3,27$

Table 1: Selectie van benchmarks. Alle waarden worden uitgedrukt in cycli.

6.2 Base

Voor de uitvoering van het programma geldt de volgende invariant:

$$\sum_{i=0}^{N_s} \mathrm{PICS}_{b,i} = N_d \cdot \frac{1}{dispatchbreedte} \tag{3}$$

N_s stelt het aantal statische instructies in het programma voor, N_d het totaal aantal dynamisch uitgevoerde *µops*, beide verkregen uit de output van de *Sniper* simulator. $\mathrm{PICS}_{b,i}$ stelt de basiscomponent van de i-de statische instructie voor zoals gedefinieerd in formule (1).

Dit is zo omdat elke *µop* steeds *gedispatcht* wordt, en dus altijd een deel van de cyclus toegeschreven krijgt aan de *base*-component. Bij constructie, zoals uitgelegd in formule (1), bedraagt dit precies $1/dispatchbreedte$. Tabel 2 vergelijkt de som over alle *PICS* van enkele benchmarks met het aantal uitgevoerde *µops* uit de simulatoroutput. Δ_c is groter dan Δ_d doordat enkele *µops* nog niet gedetaileerd zijn op het moment dat de simulator afsluit (zie sectie 9).

Benchmark	uops/breedte	$base_d$	Δ_d	$base_c$	Δ_c
ED1	5.122,75	5.120,70	$-2,05$	5.098,46	$-24,29$
STL2b	50.242,50	50.240,14	$-2,36$	50.214,10	$-28,40$
MCS	32.773,00	32.770,68	$-2,32$	32.755,49	$-17,51$
MI	30.346,50	30.344,75	$-1,75$	30.331,26	$-15,24$
DPcvt	3.602,50	3.600,50	$-2,00$	3.583,47	$-19,03$
EI	4.626,25	4.624,25	$-2,00$	4.622,21	$-4,04$
CCl	4.111,00	4.109,24	$-1,76$	4.097,22	$-13,78$

Table 2: Selectie van benchmarks. Alle waarden worden uitgedrukt in micro-operaties per dispatch-slot.

6.3 Frontend stall en Drained

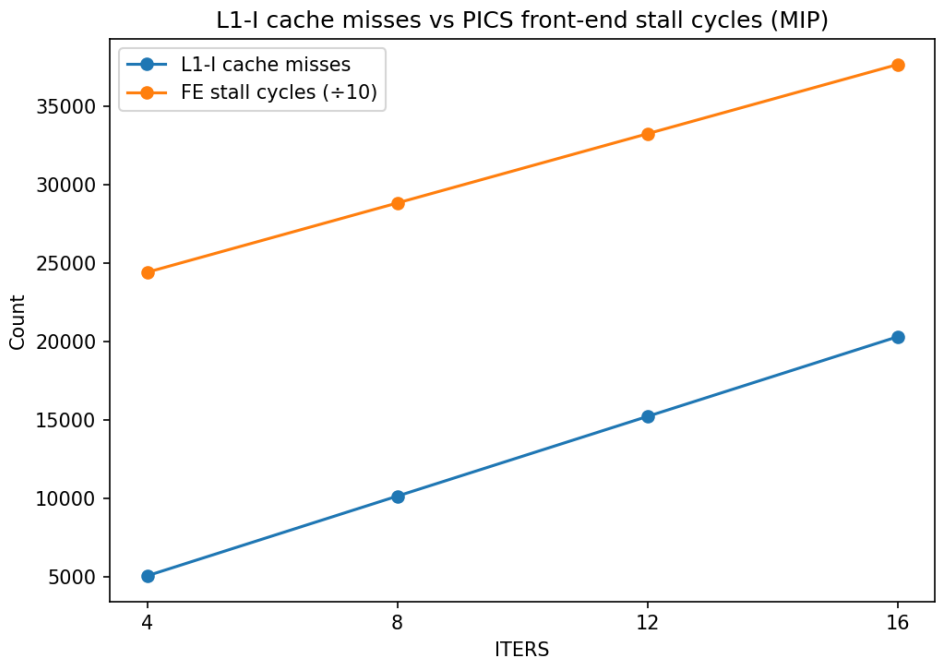
Het aantal cycli dat toegeschreven wordt aan de component *frontend stall* is bijna uitsluitend afkomstig van *I-cache misses*. Om deze component te verifiëren wordt er gekeken naar een *microbenchmark* met veel *I-cache misses*, waarbij het aantal herhalingen van de lus wordt opgeschaald. Het verwachte resultaat is dat door het opschalen van het aantal iteraties er meer *I-cache misses* zijn wat op zijn beurt zou moeten leiden tot een groter aantal cycli toegeschreven aan de *frontend stall* componenten.

De gebruikte *microbenchmarks* bestaat uit een lange reeks (ongeveer 11 000) bewerkingen op een variabele `t`, waarbij dezelfde reeks meerdere keren na elkaar is uitgeschreven. Hierdoor wordt de lus in machinecode fysiek groot genoeg om *I-cache misses* te veroorzaken.

```
...
for(iter=0; iter < ITERS; ++iter) {
    t += 1;
    t *= 2;
    t -= 1;
    t &= 0xFFF00FFF;
    t ^= 0x14022;
    t %= 0xFFFFFF3;
    t += 6;
    t *= 10;
    ...
    t %= 0xFFFFFF3;
    t += 6;
    t *= 10;
}
```

C-code van de MIP benchmark

Het analyseren van de benchmark voor verschillende aantal iteraties resulteert in Figuur 4. Het is duidelijk dat het verwachte gedrag bij dispatch wordt bevestigd.



Figuur 4: MIP voor verschillend aantal iteraties

Bij *commit* is het schaalgedrag niet zichtbaar. De *frontend stalls* die bij *dispatch* zichtbaar zijn en schalen met het aantal *I-cache misses* worden gemaskeerd bij *commit*. Dit maakt het verifiëren van de *drained* component lastig. Er kan uiteindelijk geconcludeerd worden dat als de som van het totaal aantal geattribueerde cycli klopt en de drie andere componenten correct zijn, de *drained* component ook correct moet zijn. Dit is een argument voor het correct zijn van het totaal aantal *drained* cycli, niet of de cycli naar de correcte instructies gaan. Het correct zijn van de drie andere componenten wordt nu verder aangetoond.

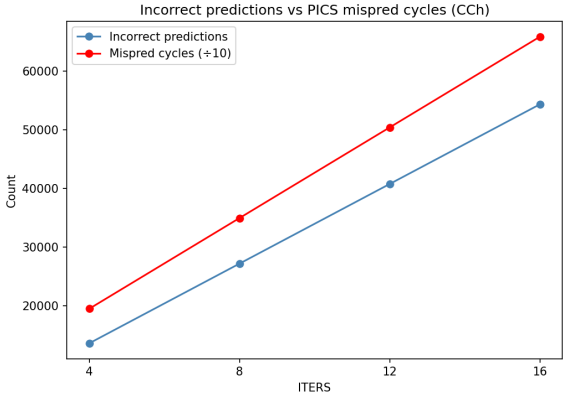
6.4 Misprediction

Het aantal cycli dat toegeschreven wordt aan de component *misprediction* is afkomstig van foute voorspellingen. Om deze component te verifiëren wordt er gekeken naar een *microbenchmark* met veel foutieve *sprongvoorspellingen*, waarbij het aantal iteraties opnieuw wordt opgeschaald. Het verwachte resultaat is dat het opschalen van het aantal iteraties leidt tot meer foutieve sprongvoorspellingen wat op zijn beurt leidt tot meer cycli toegeschreven aan de *misprediction* componenten.

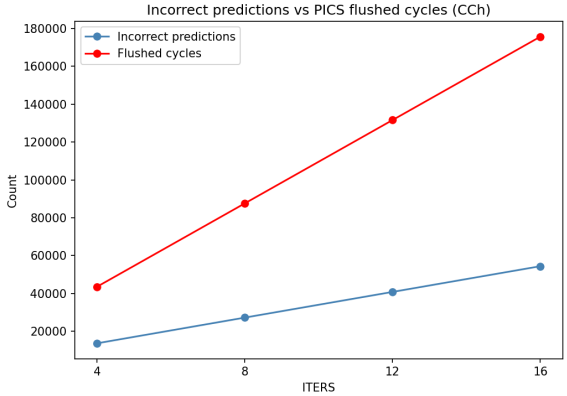
De gebruikte *microbenchmarks* bestaat uit een lus met een willekeurige sprongconditie, waardoor er dus veel foute voorspellingen zullen zijn.

```
...
for(i=0; i < STEP + zero; i+=1) {
  if(randArr[i])  {
    t+=3+3*t;
  } else {
    t-=1-5*t;
  }
}
...
```

C-code van de CCh benchmark



Figuur 5: CCh dispatch voor verschillend aantal iteraties.



Figuur 6: CCh commit voor verschillend aantal iteraties.

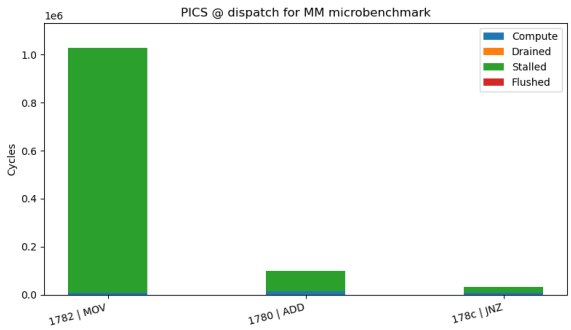
Figuur 5 toont het schaalgedrag van de *microbenchmark* bij *dispatch*. Het verwachte gedrag, meer foutieve sprongvoorspellingen leidt tot meer *misprediction* cycli, wordt hier duidelijk zichtbaar. Figuur 6 toont het schaalgedrag van de *microbenchmark* bij *commit*. Wat opvalt is dat het aantal *flushed* cycli bij *commit* minder is dan bij *dispatch*. Dit is te wijten aan het feit dat de gevolgen van de foutief voorspelde sprong gemaskeerd worden onder *backend stalls* bij *commit* (zie sectie 7.1). In het algemeen zijn er twee mogelijkheden hoe de gevolgen van foutief voorspelde sprongen zichtbaar worden bij *commit*: ze worden niet of weinig gemaskeerd, waarvoor hier aangetoond werd dat het gedrag correct is. Of ze worden (bijna) volledig gemaskeerd waardoor het correcte gedrag van *backend stalls* ervoor zorgt dat ook dit gedrag correct is.

6.5 Backend stall en stalled

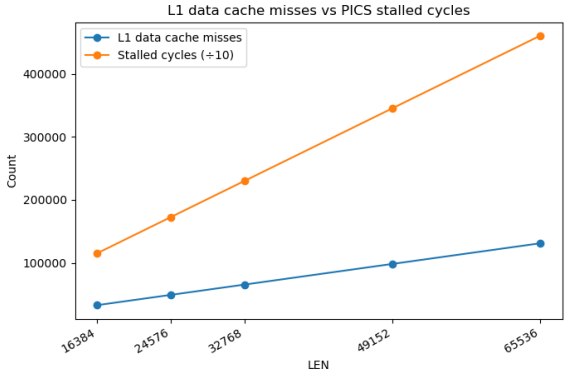
Analoog aan de *drained* component van *PICS* bij *commit*, wordt het argument voor de verificatie van de *backend stall* component van *PICS* bij *dispatch* afgeleid uit de andere onderbouwingen. Er is reeds een argument voor de *base*, *frontend stall* en *misprediction* component van *PICS* bij *dispatch*, alsook de totale som. Logischerwijs zijn de cycli uit

de som die niet voorkomen in één van de drie componenten, *backend stall* cycli. Dit geeft dus enkel een argument voor het totaal aantal geattribueerde *backend stall* cycli, maar zegt niets over of de cycli naar de juiste instructie gaan.

Voor de *stalled* component van *PICS* bij *commit* wordt gebruik gemaakt van een analoge redenering als bij de *frontend stall* component van bij *dispatch*, zoals beschreven in sectie 6.3. De *MM microbenchmark* loopt een *gelinkte lijst* af. Elk element bevat, naast zich in het geheugen, ook het adres van het volgende element. Daarnaast wordt ervoor gezorgd dat elk element een volledige *cachelijn* - het aantal bytes dat in één keer uit het geheugen geladen wordt en in de *cache* opgeslagen wordt - inneemt. Elke *load* van een element moet dus naar het hoofdgeheugen én elke *load* is afhankelijk van de vorige voor het adres. De trage *load*-instructie zal dus telkens vastzitten als oudste instructie in de *ROB*, wat zal leiden tot cycli op de *stalled* component van zijn *PICS* bij *commit*, zoals geïllustreerd in Figuur 7.



Figuur 7: PICS bij commit voor de MM microbenchmark.



Figuur 8: Stalled cycli en datacache misses in functie van de lijstlengte.

Als de kritieke *load*-instructie nu meer uitgevoerd wordt - door de *gelinkte lijst* langer te maken - zou het logische gevolg moeten zijn dat het aantal *stalled* cyli eveneens toeneemt. Figuur 8 toont het aantal *datacache misses*, zoals gerapporteerd door de *Sniper* simulator, samen met het totaal aantal geattribueerde *stalled* cycli over alle *PICS*. Merk op dat dit lineair gedrag zich niet zou blijven voortzetten. Bij een bepaalde grootte zal de *gelinkte lijst* de grootte van de eerste niveau *cache* overschreiden, en zal de *load*-instructie bij elke uitvoering meer *stalled* cycli bekomen op zijn *PICS* bij *commit*.

7 Resultaten

Na het verifiëren van de *PICS* kunnen enkele codefragmenten geanalyseerd en eventueel geoptimaliseerd worden. In wat volgt worden twee zulke fragmenten besproken, die tevens aantonen dat *PICS* bij *dispatch* en *PICS* bij *commit* een complementair beeld geven.

7.1 Optimalisatie van PSum

Als eerste programma wordt PSUM gekozen. Een *array* wordt partieel opgeteld gedicteerd door een willekeurig gegenereerde *array*. De kern van het programma wordt gegeven door het volgende fragment:

```
...
for (i = 0; i < ITER + zero; ++i) {
    if (randArr[i]) {
        index = (i * STRIDE) % ARRAY_SIZE;
        t += data[index];
    }
}
...
```

C-code van PSum

Bij de uitvoering van deze code wordt voornamelijk een prestatieverlies verwacht door foute sprongvoorspellingen, aangezien de sprongconditie afhankelijk is van een willekeurige waarde. Om te bepalen welke instructie hier exact verantwoordelijk voor is, wordt de gegenereerde assembly-code geanalyseerd:

```
...
401780:     cmpb    $0x0,(%rcx)
401783:     je      40179f <loop+0x5f>
401785:     mov     %eax,%edx
401787:     imul    %r9,%rdx
40178b:     shr     $0x22,%rdx
40178f:     lea     (%rdx,%rdx,4),%r8d
401793:     mov     %eax,%edx
401795:     sub     %r8d,%edx
401798:     movslq  %edx,%rdx
40179b:     add     -0x28(%rsp,%rdx,4),%esi
40179f:     add     $0x2,%eax

...
```

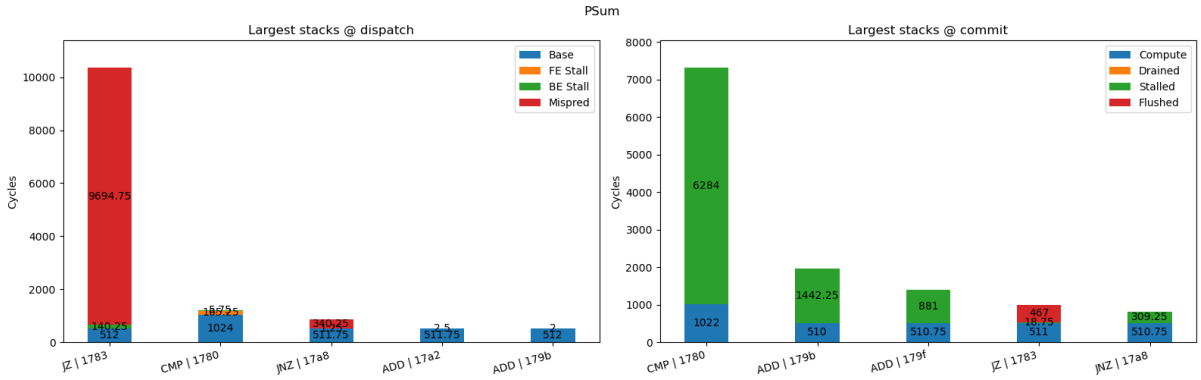
Assembly code van PSum

De interessante instructie is `je` op adres `401783` - de spronginstructie waarvan de conditie afhangt van `randArr[i]`. Indien `randArr[i]` gelijk is aan nul wordt de sprong genomen en de berekening van `t` overgeslagen. In het andere geval worden de instructies van `401785` tot `40179b` uitgevoerd, waarbij `index` berekend wordt en `t` geaccumuleerd wordt. Beide paden komen samen op `40179f`, waarna de volgende iteratie begint.

Omdat `randArr` willekeurige waarden bevat, kan de sprongvoorspeller geen patroon herkennen en zal er frequent een foute voorspelling gemaakt worden. Er wordt dan ook verwacht dat `je` de instructie is met de meeste geattribueerde cycli, voornamelijk door foutieve voorspellingen. Figuur 9 toont de *PICS* bij *dispatch* en bij *commit*. Bij *dispatch* volgt de *PICS*

de verwachtingen en wordt `je` correct aangeduid als bottleneck. Opmerkelijk is dat dit bij *commit* niet het geval is: daar wordt de bottleneck gemaskeerd door de `cmp` instructie op adres `401780`, die als geheugentoegang een *backend stall* ondergaat.

Dit illustreert het belang van het gecombineerd bekijken van *PICS* bij *dispatch* en bij *commit*, aangezien beide een complementair beeld geven en elk andere bottlenecks aan het licht brengen.



Figuur 9: PICS van de PSum benchmark

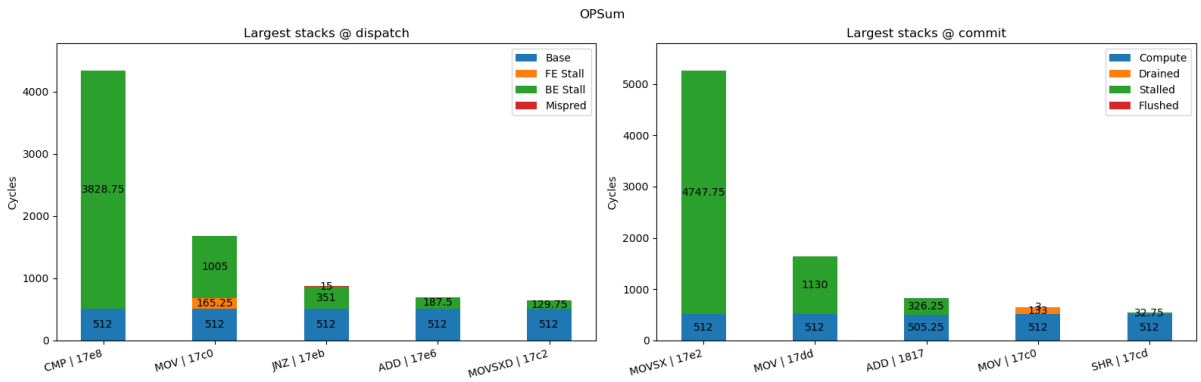
Merk op dat de *Sniper* simulator soms afwijkende namen gebruikt ten opzichte van de *assembler* - e.g. `je` en `jz` - die echter dezelfde instructie voorstellen.

Als optimalisatie wordt de sprong geëlimineerd door de berekening van `index` en de accumulatie van `t` te scheiden in twee afzonderlijke lussen:

```
...
for(i = 0; i < ITER + zero; ++i)
{
    indices[a] = (i * STRIDE) % ARRAY_SIZE;
    a += randArr[i];
}
for(i = 0; i < a + zero; ++i)
{
    t += data[indices[i]];
}
...
```

C-code van OPSum

Door de lussen te splitsen verdwijnt de conditionele sprong volledig. Het gevolg is dat er een extra doorloop nodig is, maar doordat er geen foutieve voorspelling meer gemaakt worden zal het blijken dat er toch een nettoverbetering is, namelijk van 16583 naar 15832. Dit is duidelijk zichtbaar op Figuur 10. De foutieve sprongvoorspellingen zijn volledig weggewerkt.



Figuur 10: PICS van OPSum

7.2 Optimalisatie van AA

Nu wordt er gekeken naar de volgende code:

```
...
int i;
n=malloc(sizeof(struct ll));
cur=n;
for(i=0;i<LEN;++i) {
    cur->val=i;
    cur->_next=malloc(sizeof(struct ll));
    malloc(3*64); // trash the memory
    cur=cur->_next;
}
cur->val=100;
cur->_next=NULL;
...
int t = 0,i;
struct ll* cur =n;
while(cur!=NULL) {
    t+=cur->val;
    cur=cur->_next;
}
return t;
...
```

C-code van AA

In dit voorbeeld wordt een *gelinkte lijst traversal* uitgevoerd. De knopen bevinden zich redelijk ver van elkaar in het geheugen, waardoor de *cache* snel vol raakt. Een *load* instructie op een bepaalde knoop is altijd afhankelijk van de *load* instructie van de vorige knoop. Op die manier zal elke *load* afhankelijk zijn van de *load* die op dat moment wordt

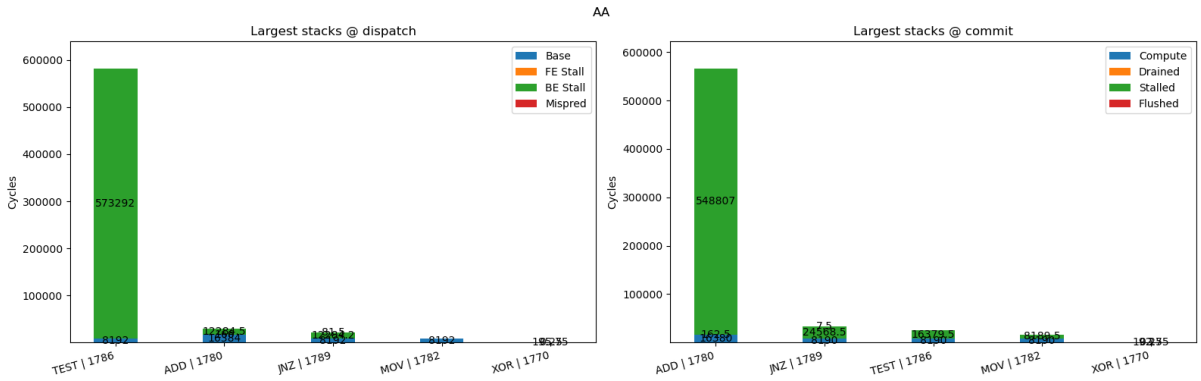
uitgevoerd. Dit betekent ook dat de *ROB* zeer snel vol raakt en er een *backend stall* zal optreden. Om de exacte instructie die de *stall* veroorzaakt te vinden, wordt er opnieuw gekeken naar de gegenereerde assembly-code.

```
...
401780:        add     (%rsi),%eax
401782:        mov     0x8(%rsi),%rsi
401786:        test    %rsi,%rsi
401789:        jne     401780 <loop+0x10>
40178b:        ret
...
```

Assembly-code van AA

De `add` op adres `401780` is de eerste instructie die een *load* uitvoert en komt overeen met de accumulatie van `t`. Daarna wordt er nog een *load* uitgevoerd om het adres van de volgende knoop te verkrijgen. Deze zal sneller verlopen, aangezien de benodigde data al in de *cache* staat dankzij de eerste *load*. Zoals eerder uitgelegd, kan de volgende knoop pas worden ingelezen wanneer de vorige *load* voltooid is, waardoor deze `add` instructie de bottleneck vormt.

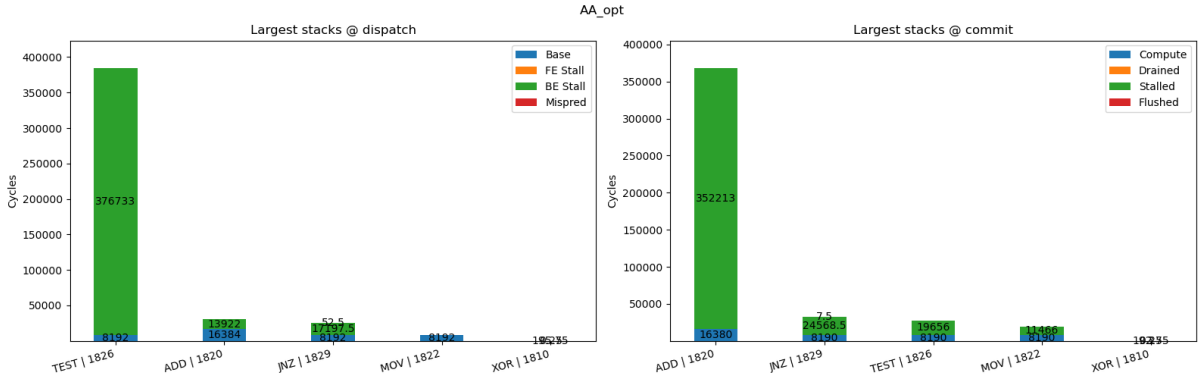
Figuur 11 toont duidelijk via de *PICS* bij *commit* dat de `add` instructie die hierboven vermeld is de bottleneck is. Merkwaardig is dat de *PICS* bij *dispatch* de `test` instructie aanduidt als bottleneck. Dit is echter te verwachten aangezien - bij constructie van de *PICS* bij *dispatch*, zie sectie 5.2 - de cycli worden geattribueerd aan de instructie die op dat moment niet kan worden *gedispatcht*. Op het moment dat de `add` instructie wacht op een *load*, staat de `test` instructie als volgende klaar om *gedispatcht* te worden en krijgt bijgevolg de cycli toegewezen. Dit toont opnieuw aan dat beide meetmomenten een complementair beeld geven en dat het visualiseren van de *PICS* bij *commit* essentieel is voor een correcte analyse.



Figuur 11: PICS van AA

Als optimalisatie wordt er een afweging gemaakt binnen het alloceren van de *tegelinkte lijst*. Er wordt gekozen om aan *arena allocation* te doen. Dit wil zeggen dat wanneer de eerste

knoop wordt gealloceerd, er onmiddellijk een groter blok wordt gealloceerd voor meerdere knopen. De volgende knopen worden dan gealloceerd door een pointer op te schuiven. Er moet echter een afweging gemaakt worden over hoeveel ruimte er wordt gealloceerd: een te groot blok reserveren verspilt mogelijks geheugen en maakt het moeilijker een vrije blok te vinden, terwijl een te klein blok weinig voordeel biedt. Als er wordt gewerkt met *arena allocation* waarbij er onmiddellijk vijf knopen worden gealloceerd, zijn de bijhorende *PICS* te zien in Figuur 12. Het aantal *backend stall* cycli daalt van 573292 naar 376733, een daling van 34%. Het initieel alloceren van meer knopen biedt echter een beperkt voordeel, aangezien de *cachegrootte* een natuurlijke grens oplegt.



Figuur 12: PICS van AAOpt

8 Reflectie

De samenwerking, organisatie en communicatie binnen de groep verliep zeer vlot. Alle studenten waren gemotiveerd en betrokken, waardoor taken efficiënt verdeeld en opgevolgd konden worden. Ook de promotor en begeleiders waren nauw betrokken bij het project. Er was wekelijks overleg mogelijk, waarbij voortgang besproken werd en ruimte was voor inhoudelijke discussie. Deze open communicatie zorgde ervoor dat problemen snel gesignaleerd en aangepakt konden worden, wat sterk bijdroeg aan het uiteindelijke resultaat.

9 Openstaande problemen

Ondanks de correcte implementatie van *PICS* in de *Sniper* simulator, zijn er nog openstaande problemen die er voor zorgen dat met bepaalde zaken geen rekeningen wordt gehouden bij het genereren van *PICS*. *PICS* toont dus een geabstraheerde versie van de realiteit. De problemen worden hieronder besproken.

9.1 Verloren instructies

Om de simulatiesnelheid te verhogen voor grote aantallen instructies kan de *Sniper* simulator enkel het *cachegedrag* van instructies simuleren in *fast forward mode*. Het deel waarin de gebruiker effectief geïnteresseerd is, gemarkeerd als *region of interest (ROI)*, wordt dan uitvoerig gesimuleerd in *detailed mode*.

Dit heeft echter gevolgen voor de implementatie van *PICS*. Op het einde van de *ROI* schakelt de simulator terug over op *fast forward mode* onafhankelijk van de staat van de *ROB*. De *PICS* bij *commit* van de instructies die zich nog in de *ROB* bevinden zullen dus niet volledig zijn. Ook het overschakelen van *fast forward mode* naar *detailed mode* in het begin van de *ROI* leidt tot verlies van nauwkeurigheid. Het programma dat de instructies uit de *binary* extraheert werkt losgekoppeld van de simulator zelf. Er zijn al een reeks instructies uit de *ROI* ongedetailleerd verwerkt voordat het extractieprogramma op de hoogte is dat het instructies in detail moet verwerken. Voor die instructies heeft de simulator dus te weinig informatie om *PICS* op te stellen. De impact is echter klein, omdat het niet schaalt met de grootte van het programma.

9.2 TLB misses

TLB-misses worden apart van de reguliere instructiestroom verwerkt en omzeilen de modellering van de *ROB* volledig. Omdat het attribueren van de *PICS* gebeurt in de code die de *ROB* modelleert, worden de cycli van een *TLB-miss* aan geen enkele instructie toegewezen.

In sectie 6.5 werd gesproken over een correctie op het totaal aantal cycli dat gerapporteerd werd door de *Sniper* simulator waarmee de som over alle *PICS* vergeleken werd. Die correctie is de som van de cycli die gesimuleerd werden voor de verloren instructies en de *TLB misses*.

9.3 DVFS

Dynamic voltage and frequency scaling (DVFS) is een techniek waarbij een processor zijn klokfrequentie (en bijhorende voedingsspanning) dynamisch aanpast om energieverbruik te beperken zonder onnodig prestatieverlies. De *Sniper* simulator kan dit modelleren. Desondanks wordt er bij de huidige implementatie van *PICS* uitgegaan van een constante klokperiode. Bij elke aanrekening van een cyclus wordt een klokperiode geteld. Het totaal wordt dan op het einde van de simulatie éénmalig door de klokperiode gedeeld om het aantal cycli te bekomen.

10 Besluit

In dit project werd PICS succesvol geïmplementeerd in de Sniper simulator, zowel bij dispatch als bij commit. De implementatie maakt het mogelijk om cycli tijdsproportioneel toe te wijzen aan statische instructies en zo inzicht te krijgen in waar uitvoeringstijd verloren gaat.

Bij dispatch worden cycli onderverdeeld in de componenten base, frontend stall, backend stall en misprediction. Bij commit gebeurt dit via compute, stalled, drained en flushed. Voor elke stall wordt de volledige latentie correct geattribueerd aan de verantwoordelijke instructie.

De implementatie werd geverifieerd aan de hand van microbenchmarks en invarianten. Daaruit bleek dat het totaal aantal geattribueerde cycli overeenkomt met de simulatieduur en dat de verschillende componenten het verwachte gedrag vertonen.

Daarnaast tonen de resultaten aan dat PICS bij dispatch en commit een complementair beeld geven van prestatieproblemen. Afhankelijk van het meetmoment kunnen andere bottlenecks zichtbaar worden. Hierdoor is het belangrijk om beide vormen van profilering samen te gebruiken bij prestatieanalyse.

De implementatie biedt ontwikkelaars een manier om prestatieproblemen in meerdradige applicaties gerichter te analyseren en optimaliseren. Hoewel er nog openstaande problemen zijn, zoals verloren instructies, TLB misses en DVFS, vormt dit project een werkende basis voor verdere uitbreiding van PICS binnen de Sniper simulator.

Referenties

[1] J. L. Hennessy, D. A. Patterson, and C. Kozyrakis, Computer Architecture: A Quantitative Approach, 7th ed. Morgan Kaufmann, 2025

[2] B. Gottschall, L. Eeckhout, and M. Jahre. 2023. TEA: Time-Proportional Event Analysis. In Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA '23), June 17–21, 2023, Orlando, FL, USA. ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3579371.3589058

[3] S. H. Campelo de Santana, J. Rogers, L. Eeckhout, and M. Jahre, "Chips Need DIP: Time-Proportional Per-Instruction Cycle Stacks at Dispatch," in Proc. 31st ACM Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS), Pittsburgh, PA, USA, Mar. 2026, pp. 361–376.

[4] T. E. Carlson, W. Heirman, S. Eyerman, I. Hur, and L. Eeckhout, "An Evaluation of High-Level Mechanistic Core Models," ACM Trans. Archit. Code Optim., vol. 11, no. 1, article 5, Apr. 2014, doi: 10.1145/2629677.

[5] T. E. Carlson, W. Heirman, and L. Eeckhout, "The Sniper Multi-Core Simulator," snipersim.org. [Online]. Available: https://snipersim.org. [Accessed: Apr. 12, 2026].

[6] J. M. Anderson, L. M. Berc, J. Dean, S. Ghemawat, M. R. Henzinger, S.-T. A. Leung, R. L. Sites, M. T. Vandevoorde, C. A. Waldspurger, and W. E. Weihl, Continuous profiling: where have all the cycles gone?, ACM Trans. Comput. Syst., vol. 15, no. 4, pp. 357–390, Nov. 1997, doi: 10.1145/265924.265925.

[7] Arm. 2017. ARM Architecture Reference Manual Supplement Statistical Profiling Extension, for ARMv8-A. `https://static.docs.arm.com/ddi0586/aDDI0586A_Statistical_Profiling_Extension.pdf`.

[8] J. Dean, J. E. Hicks, C. A. Waldspurger, W. E. Weihl, and G. Chrysos. 1997. ProfileMe: Hardware Support for Instruction-Level Profiling on Out-of-Order Processors. In Proceedings of the International Symposium on Microarchitecture (MICRO). IEEE Computer Society, 292–302.